שאלה 1

הגדרה: זוג (A, a) של מערך A של מספרים ממשיים ומספר ממשי a נקרא ממוין גדולים אם"ם כל אברי A הגדולים מ-a ממוקמים במערך בצורה ממוינת, ואברי המערך הקטנים שווים ל- a נמצאים במערך בצורה לא בהכרח ממוינת.

דוגמה: הזוג $([1,8,3,10,2,12,1,14,0,5], 7)$ הוא ממוין גדולים.

הוכיחו שלא קיימת פונקציה המקבלת מערך B של מספרים ממשיים וכן מספר ממשי b , מבצעת אך ורק פעולות השמה והשוואה, פועלת בסיבוכיות קטנה ממש מ- $\boldsymbol{\theta(n\log n)}$ ומחזירה מערך C שהוא תמורה של B והזוג (C, b) הוא ממוין גדולים.

הוכחה: נניח בשלילה קיים אלגוריתם big שמקבל מערך B של מספרים ממשיים וכן מספר ממשי b וממיין בביצוע פעולות השוואות והשמות בסיבוכיות קטנה $\theta(n\log n)$.

פעולות	סיבוכיות	
השמות והשוואות	$\theta(n)$	1)
השמות והשוואות	$<\theta(n\log n)$	2)
השוואות והשמות	$<\theta(n\log n)$	סה"כ

```
sort(B)
1) b ← min(B)-1
2) C ← big(B,b)
return(C)
```

מסקנה: בגלל Sort מבוסס השוואות בסיבוכיות קטנה מ- $\theta(n\log n)$ בהנחה למשפט כל מיון מבוסס השוואות והשמות חייב לפעול בסיבוכיות גדול/שווה $\theta(n\log n)$ לכן לא קיים אלגוריתם big מבוסס השוואות והשמות שפועל בסיבוכיות קטנה $\theta(n\log n)$

מ.ש.ל

2) א. בשיטת התכנות דינמי נשתמש ב 2 מערכים דו-ממדיים ND, WD , n×n+2

ND[i,j] - הסכום המקסימלי של i איברים במסלול A ל- i

השורות העליונות כשחייב לקחת את A[i,j] ואופן שבמעבר

האחרון (ל- A[i,j]) יהיה אלכסוני (לפי הגדרת השאלה)

WD[i,j] - הסכום המקסימלי של i איברים במערך A ל- i

השורות העליונות כשחייב לקחת את A[i,j] ומיד שהמעבר

האחרון (ל- A[i,j]) יהיה אלכסוני (לפי הגדרת השאלה)

```
MaxSumPath(A)
  for j ← 1 to n
    ND[1,j] ← WD[1,j] ← A[1,j]
  end for
  for i ← to n
    ND[i,0] ← WD[i,0] ← ND[i,n+1] ← WD[i,n+1] ← -∞
  endfor
  for i ← 2 to n
    for j ← 2 to n
      ND[i,j] ← A[i,j] + max(ND[i-1], WD[i-1,j]
      WD[i,j] ← A[i,j] + max{ND[i-1,j], WD[i-1,j],
                              ND[i-1,j-1], ND[i-1,j+1]}
    endfor
    return (max(max(nd[n,1....n]), max(WD[n,1....n])))
```

ב) תחילה הדיתקנו את השורה הראשונה של מערך הקלט לשורה הראשונה של DM ו-WD. נשים לב שכל חצי מ-1 הכניסות DM[1,j] ו- WD[1,j] מתקיימות לאחר מכן נביט בשני מערכי המשנה הללו ל [-∞, 0] המסומנות 0 ו- h+1

כאשר ניגש למילוי DM ו-WD כך שהכניסות יתקיימו באינדוקציה: לכל איבר A[i,j] אשר שהמסלול האחרון במסלול יהיה מסבר אלכסוני נחייב לקחת את A[i,j] לכן נינתן לתבר את A[i,j] לפרוס של מסלול שנגמר באיבר שמעליו (A[i-1,j]).

נשים לב שלפי אילוצי השאלה במסלול שבנה המסבר האחרון יכול להיות או אלכסוני או ישר (נבחר את מה שתרומתו מירבית לפרוס). התקיימות הנכונות DM נבדקת.

לכל איבר A[i,j] מותר שהמסבר האחרון במסלול יהיה מסבר אלכסוני נחייב לקחת את A[i,j] לכן ניתן לחבר את A[i,j] לפרוס של המסלולים הבאים:

מסלול שנגמר ב-A[i-1,j] ללא הגבלה על המסבר האחרון (אפשר לקחת אלכסוני או ישר כי החיבור הנוכחי לא משבש)

מסלול שנגמר ב-A[i-1,j-1] או A[i-1,j+1] שלו המסבר האחרון האחרון לא אלכסוני כי החיבור הנוכחי הוא אלכסוני ולכן קיימת הגבלה. נבחר את המסלול שתרומתו

לסכום מירבית. התקיימות הגדרת סש נבדקת.
מכאן נובע שאחד האיברים בשורה ה-n כ-סמ
(המקסימלי) או אחד האיברים בשורה ה-n בסש (המקסימלי)
מייצג את הפלט הרצוי של השאלה נבחר את הגדול מבין
השניים) וזה מה שמחזיר בסוף האלגוריתם

Q – הסיבוכיות השורה הראשונה של A עסש ו-סמ

$$\Theta(2n) = \Theta(n)$$

– רקורסיה ∞ – בסמוכות ס ו-1 ו-n

$$\Theta(4n) = \Theta(n)$$

– מילון סש ו-סמ

$$\Theta(n^2)$$

– התרת המקסימום בין 2 המקסימלים בשורה
ה-n כ-סמ, סש

$$\Theta(2n) = \Theta(n)$$

<u>סה"כ</u>

$$T(n) = \Theta(n) + \Theta(n) + \Theta(n^2) + \Theta(n) + c = \Theta(n^2)$$

L3

```
BS5(A)
B ← A
if(B[1]>B[2]) swap(B[1],B[2])
if(B[2]>B[3]) swap(B[2],B[3])
if(B[3]>B[4]) swap(B[3],B[4])
if(B[4]>B[5]) swap(B[4],B[5])
if(B[1]>B[2]) swap(B[1],B[2])
if(B[2]>B[3]) swap(B[2],B[3])
if(B[3]>B[4]) swap(B[3],B[4])
if(B[1]>B[2]) swap(B[1],B[2])
if(B[2]>B[3]) swap(B[2],B[3])
if(B[1]>B[2]) swap(B[1],B[2])
out(B)
```

סדרת ההשוואות המשורה עבור כתמורת הקלט $A=(4,2,3,5,1)$

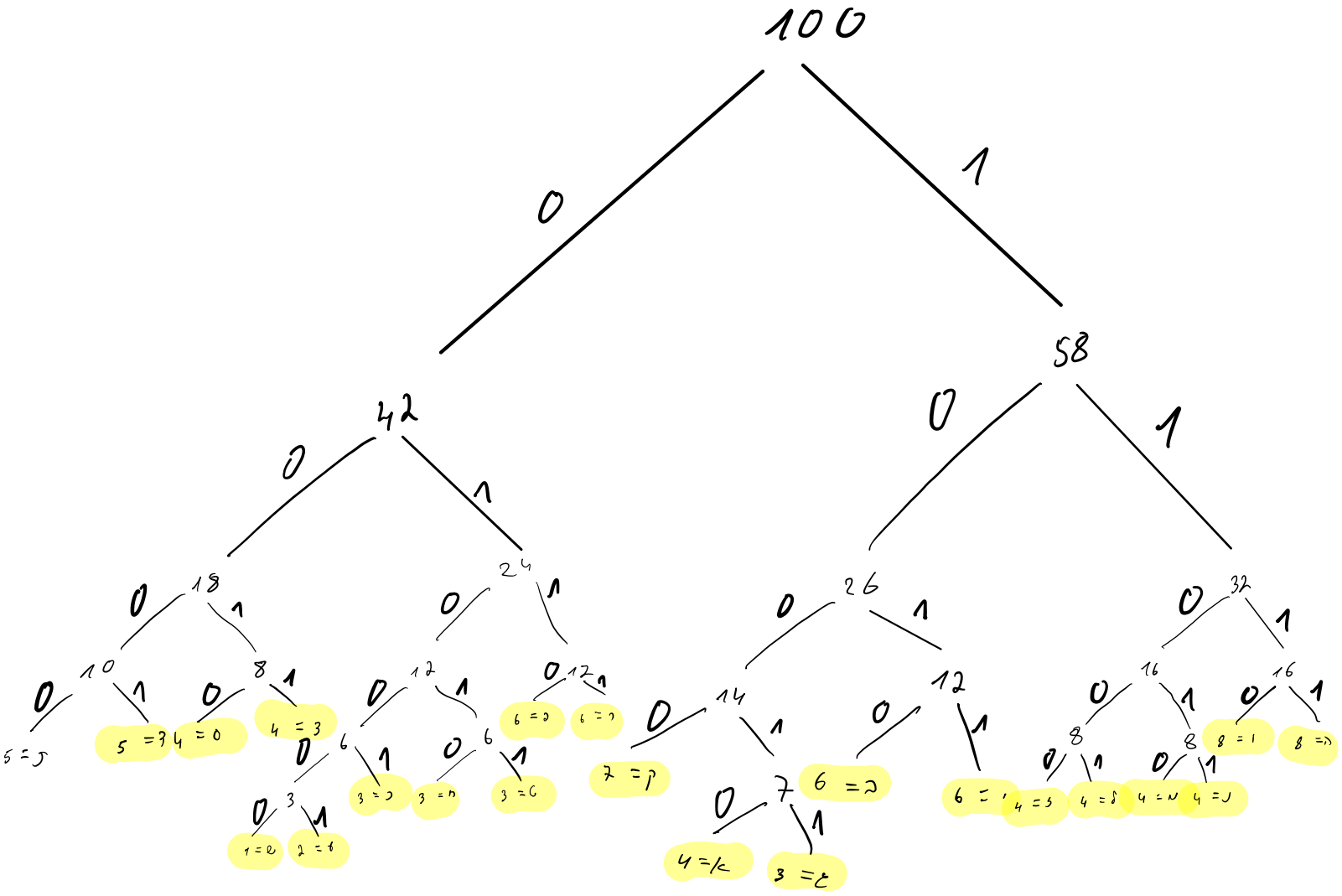
$(a_1>a_2,t),(a_2>a_3,t),(a_3>a_4,F),(a_4>a_5,t),(a_1>a_2,F),$
$(a_2>a_3,F),(a_3>a_4,t),(a_1>a_2,F),(a_2>a_3,t),(a_1>a_2,t)$

C4

1110 = ן
000010 = ל
0000 = ן

1101 = ן
10011 = ם
01011 = נ
11011 = ן
0100 = ס
100010 = ת
0110 = ן
1100 = ג
0001 = ף

010001 = ק
0101 = ן
11001 = ץ
1000 = ז
1111 = ן
0111 = ן
00011 = ס
01010 = ן
11010 = '



5) מערך מספרים טבעיים מגודל n יקרא ממוין שלש אם כל איבריו באינדקס שמתחלק ב-3 ממוקמים בצורה ממוינת עולה ושאר האיברים מצויים במקומותיהם בסדר כלשהו. הוכיחו שאי אפשר להפוך מערך ממוין שלש לבין בידוד רק בעזרת השוואה והשמה תוך בידוד בחוות ב-$\Theta(n\log n)$ פעולות.

הוכחה: נניח בשלילה שקיים אלגוריתם Three שממיין מערך בפעולות השוואה והשמה בפחות ב-$\Theta(n\log n)$ פעולות.

```
sort(A)
1) B[1,2,.....,3n] ← [1,.....,1]
2) B[3,6,.... 3n-3,3n] ← A[1,2,....n]
3) B ← Three(B)
4) out(B[3,6,.....,3n-3,3n]).
```

	סיבוכיות	פעולות
1)	$\Theta(3n)$	השמה
2)	$\Theta(n)$	השמה
3)	$\Theta(3n\log 3n) = \Theta(n\log n) >$	השמות והשוואות
סה"כ	$\Theta(n\log n) >$	השמות והשוואות

מסקנה: בידוד נו sort בסיבוכיות $\Theta(n\log n) >$ מבוסס השוואות והשמות בסתירה לנשפט: כל אלגוריתם מבוסס השוואות והשמות חייב לפעול בסיבוכיות $\Theta(n\log n) \geq$ לכן לא קיים אלגוריתם Three מבוסס השוואות הפועל בסיבוכיות $\Theta(n\log n) >$

מ.ש.ל

א. קוד תמורות: קוד בינארי בן לכל שני תווים c_1 ו- c_2 והקוד של c_1 אינו תמורות של הקוד c_2.

ב. קוד חד-פסגה: קוד בן לכל 2 קבדים F_1, F_2 כש $F_1 \neq F_2$ מתקיים: $K(F_1) \neq K(F_2)$

ג. נשאל את הטענה שכל קוד תמורות הוא חד-פסגה:
לפי הגדרת קוד תמורות שנמצאת בסעיף א, קוד תמורות הוא קוד שמבטיח שאין בו שני תווים עם קידוד שבו אחד הוא תמורות של השני. כלומר כאשר קוד הוא קוד תמורות ונרצה לבדוק בפענוח את תמורות וסיומו של הקידוד של כל תו. במילים אחרות כל קובץ מפוענח בדורה אחת וחד משמעית ולכן עבור שני קבדים שונים בהכרח יהיו 2 קידודים שונים.